

Securing AI-Generated Code

Why Deterministic Tools Must Establish the Baseline

Dawei Feng · Agent Infrastructure



Abstract

Vibe coding tools have significantly reduced the barriers to developing full-stack web applications. Many of these platforms offer built-in "security scans," typically performed by large language models (LLMs) that analyze the generated code. However, the effectiveness and reliability of such AI-driven security reviews remain uncertain.

In this article, we present a case study comparing AI-based security scans from two of our competitors, A and B, with Replit's **hybrid approach**, in which **deterministic scanners perform primary detection** and an LLM assists with remediation and higher-level reasoning. We use a set of intentionally vulnerable React applications for this analysis. Using identical prompts and controlled code variations, we evaluate each platform's effectiveness at detecting common frontend security and privacy issues, including cross-site scripting (XSS), secret exposure, misconfigurations, and vulnerable dependencies.

Our findings show that AI-only scans are inherently **nondeterministic and brittle**, often missing vulnerabilities depending on syntactic form or prompt phrasing. In contrast, rule-based static analysis and dependency scanning provide **deterministic, repeatable detection** with significantly broader coverage.

We conclude that LLMs are best used as remediation assistants, not security oracles, and that effective security scanning for AI-generated code requires explicit, rule-driven analysis.

1. Introduction

LLMs can now generate React applications, including UI components, backend APIs, and database integrations in minutes. This shift raises a question: *who is responsible for security when code is generated by AI?*

Several vibe coding platforms address this concern by offering integrated "security scans." In practice, these scans are typically implemented as LLMs instructed to analyze generated code and identify potential issues. While appealing, this approach implicitly asks the model to audit its own output.

Security engineering has long relied on deterministic tools: static analysis, dependency scanning, and taint tracking, precisely because PR reviews are inconsistent. This work explores whether LLM-based security reviews can replace or meaningfully approximate those tools.

We focus on frontend React applications, where vulnerabilities such as XSS, secret leakage, and unsafe dependency usage are common and well-understood.

Approach	AI-Only Scanning (LLM)	Rule-Based Scanning (SAST / SCA)	Hybrid Approach
Detection method	Probabilistic interpretation	Deterministic pattern matching	Deterministic detection + LLM reasoning
Consistency	Nondeterministic	Deterministic	Deterministic baseline
Prompt sensitivity	High	None	Low
Syntactic robustness	Fragile	Robust	Robust
Dependency CVE detection	✗	✓	✓
Supply-chain awareness	✗	✓	✓
Business-logic reasoning	⚠ Possible but inconsistent	✗	✓
Role of LLM	Primary detector	Not used	Remediation + logic analysis

Table 1. Comparison of security scanning approaches for AI-generated code.

2. Experimental Methodology

2.1 Application Generation

We defined a minimal React "micro-blog" application specification designed to resemble common AI-generated frontend applications. The specification emphasized rapid development, rich text rendering, and minimal configuration, while intentionally omitting explicit security requirements.

Using the same prompt, we asked each platform's AI agent to generate a baseline implementation of the application. This baseline included React components, dependencies, and any backend or storage logic provided by the platform.

2.2 Controlled Code Variants

To evaluate scanner robustness across realistic security scenarios, we introduced small, localized code variants to the generated applications. These edits were intentionally minimal, preserved application functionality, and reflect common real-world user behavior (e.g., temporary debugging changes).

Examples of such variants include:

- ⚠ Inserting hardcoded API keys directly into React components
- ⚠ Rendering secrets via JSX literals or variable references

Design Note: Manual variants were necessary because modern LLMs have learned best practices like storing credentials in environment variables. Secret exposure often occurs during debugging or rapid prototyping—scenarios we needed to represent.

```
8   const value = "2c48da94029b0ea566ac3df604803767"
9   return (
10    <motion.div
11      initial={{ opacity: 0, y: 20 }}
12      animate={{ opacity: 1, y: 0 }}
13      transition={{ duration: 0.4, delay: index * 0.1 }}
14    >
15      <Card className="border-0 shadow-md hover:shadow-xl transition-all duration-300 bg-white ove
16        <CardHeader className="pb-3">
17          <div className="flex items-start justify-between">
18            <h2 className="text-xl font-semibold text-gray-900 group-hover:text-indigo-600 transit
19              {post.title}
20            </h2>
21          </div>
22          <div>debug: {value}</div>
23
```

Figure 2: Example of a controlled code variant with an exposed API key

2.3 Vulnerability Classes Evaluated

- 1 Stored cross-site scripting (XSS)
- 2 Secret exposure via variable assignment

3 Secret exposure via JSX literal rendering

4 Vulnerable frontend dependencies (CVEs)

2.4 Evaluation Criteria

For each platform and variant, we measured:

- ✓ **Detection coverage** — whether the issue was reported at all
- ✓ **Precision** — whether reported issues reflected real risk
- ✓ **Consistency** — whether repeated scans produced the same results
- ✓ **Specificity** — whether findings identified concrete causes
- ✓ **Remediation support** — whether actionable fixes were proposed
- ✓ **Trials (n = 10)** — for each platform and variant, we ran the security scan 10 times on identical source code to measure nondeterminism

3. Results

3.1 Stored XSS via `dangerouslySetInnerHTML`

In the baseline generated applications, all platforms produced React components that rendered user-controlled HTML using `dangerouslySetInnerHTML`.

```

38     <CardContent className="pt-0">
39         <div
40             className="prose prose-sm max-w-none text-gray-700
41                 prose-headings:text-gray-900 prose-headings:font-semibold
42                 prose-a:text-indigo-600 prose-a:no-underline hover:prose-a:underline
43                 prose-strong:text-gray-900
44                 prose-ul:my-2 prose-ol:my-2 prose-li:my-0"
45             dangerouslySetInnerHTML={{ __html: post.body }}
46         />
47     </CardContent>

```

Figure 3: React component using dangerouslySetInnerHTML

- ✓ Replit's underlying static analysis scanner reliably matched this pattern.
- ⚠ Competitor A detected this pattern when explicitly prompted, though detection was inconsistent across runs.
- ✗ Competitor B did not report this issue.

Replit Scanner	Competitor A	Competitor B
✓ Detected	✓ Detected	✗ Not detected

3.2 Secret Exposure via Variable Assignment

Variant tested:

PostCard.tsx

```

const key = "some_api_key";

return (
    ...
    <div>Debug: {key}</div>
    ...
)

```

Replit Scanner	Competitor A	Competitor B
✓ Detected	✗ Not detected	✓ Detected

3.3 Secret Exposure via JSX Literal

Variant tested:

```
PostCard.tsx

return (
  ...
  <div>Debug: some_api_key</div>
  ...
)
```

This result is particularly significant: embedding secrets directly into JSX literal text bypassed AI-only scanners entirely, while deterministic scanning caught it.

Replit Scanner	Competitor A	Competitor B
✓ Detected	✗ Not detected	✗ Not detected

3.4 Dependency Vulnerabilities

We instructed AI agents to install and use `@tanstack/react-query-next-experimental@5.17.0` with a HIGH severity vulnerability CVE-2024-24558, which could cause a Cross-site Scripting vulnerability.

This result is expected: without dependency scanning, AI-only systems cannot reason about version-level risk.

Replit Scanner

Competitor A

Competitor B

✓ Detected

✗ Not detected

✗ Not detected

Security and Privacy Scanner Beta

Run a scan to check for potential security risks and privacy leaks in your application. Scans are typically complete within minutes. [Learn more](#)

 Run scan

 Scan settings

Active Issues Hidden Issues

4 potential vulnerabilities found.
Last ran on 4:57 pm, Dec 12, 2025

 Security

Outdated dependencies have known vulnerabilities

▼

There are dependencies that have vulnerabilities and may need to be updated.

These dependencies can be updated automatically.

 @tanstack/react-query-next-experimental@5.17.0

Hide

Update automatically

Fix with Agent

 Security

A gitleaks Generic API Key detected. Avoid hardcoding credent...

^

A gitleaks Generic API Key detected. Avoid hardcoding credentials directly in connection strings as this creates security risks. Instead, use environment variables to store and access credentials at runtime. If you need the actual credential values to make this work properly, ask me to provide them through Replit's secret system.

 client/src/components/PostCard.tsx

10 `const key = "2c48da94029b0ea566ac3df604803767";`

Hide

Fix with Agent

 Security

Detection of dangerouslySetInnerHTML from non-constant de...

^

Detection of dangerouslySetInnerHTML from non-constant definition. This can inadvertently

Detection of dangerouslySetInnerHTML from non-constant definition. This can inadvertently expose users to cross-site scripting (XSS) attacks if this comes from user-provided input. If you have to use dangerouslySetInnerHTML, consider using a sanitization library such as DOMPurify to sanitize your HTML.

 client/src/components/PostCard.tsx

37 dangerouslySetInnerHTML={{ __html: post.body }}

Hide

Fix with Agent



Security

Detection of dangerouslySetInnerHTML from non-constant de... 

Hide

Fix with Agent

Figure 4: Replit's Security and Privacy Scanner detecting multiple vulnerability types

4. Analysis

4.1 Determinism vs Interpretation

Static analysis tools such as Replit's Security Scanner operate on an explicit, rule-based model: they parse source code into an abstract syntax tree (AST) and match patterns against well-defined structural rules. This approach yields **deterministic, reproducible results**. Given the same code and the same rule set, the scanner will always produce the same findings, regardless of formatting, indentation, identifier naming, or refactoring.

This determinism was clearly reflected in our experiments. Across all variants—whether a secret was embedded as a JSX literal, stored in a variable, logged to the console, or rendered via a child component—the scanner consistently identified the same class of issue.

Literal string:

```
<div>Debug: some_api_key</div>
```

Variable indirection:

```
const key = "some_api_key";  
<div>Debug: {key}</div>
```

In contrast, AI-only scanners analyze code through interpretation rather than structural pattern matching. Their judgments depend on natural-language cues, statistical associations, and perceived intent rather than on deterministic structural rules. This interpretive model makes them highly flexible, but also **inherently non-deterministic**. Functionally equivalent code can receive different classifications depending on phrasing, variable names, or even how "obvious" the vulnerability appears to the model.

This inconsistency arises because LLMs behave stochastically: two identical prompts can yield different conclusions, and small phrasing changes in code (e.g., renaming `key` to `myValue` or rearranging JSX) can alter the model's interpretation. It did not maintain a stable mapping between syntactic patterns and security findings.

🎯 4.2 Coverage vs. Prompt Sensitivity

AI-based security scanners derive much of their effectiveness from the prompts they receive. Unlike rule-based scanners, which apply a fixed set of checks uniformly, LLM-driven scanners implicitly rely on natural language prompts to determine which classes of vulnerabilities to consider. As a result, their security coverage is not stable; it varies depending on how explicitly the prompt enumerates threats, contexts, or security goals.

This behavior was evident throughout our experiments. When platforms such as Competitor A were explicitly instructed to "analyze the code for XSS and unsafe HTML rendering," the LLM often identified `dangerouslySetInnerHTML` and flagged it as a potential security risk. However, when the same code was reviewed under a more

generic prompt (e.g., "review this project for security issues"), detection became inconsistent or disappeared entirely.

Key Finding: Prompt sensitivity introduces a structural limitation for AI-only scanners: coverage is bounded by what the user (or prompt engineer) already knows to ask and by the checks implied by the system prompt.

This limitation was particularly evident in our dependency experiments. Dependency CVEs were never flagged by AI-based scanners—not as implementation bugs but as capability gaps. These platforms do not perform dependency scanning, and an LLM reviewing code alone has no reliable way to map package names + versions to known vulnerable ranges without consulting an up-to-date vulnerability source.

This gap becomes especially critical in the context of modern supply-chain attacks. For example, the recent NPM ecosystem attack, commonly referred to as Shai-Hulud, did not manifest as an obvious code-level flaw, but rather as a malicious or compromised dependency introduced through normal package installation. Detecting such attacks requires ongoing monitoring of package metadata, version histories, and vulnerability disclosures, which reside outside the application code.

More fundamentally, CVE awareness is not a stable property of an LLM. Even if a model has seen some vulnerability disclosures during training, it will not reliably encode the full vulnerability corpus, including affected version ranges and remediation guidance. And because training data is inherently time-bounded, a model cannot be expected to react to newly disclosed CVEs without an external, continuously updated vulnerability feed.

From a systems perspective, prompt-sensitive coverage shifts responsibility away from the tool and onto the user. Effective use of AI scanners requires security expertise, careful prompt construction, and often multiple iterative passes to approach comprehensive coverage. This workflow is fundamentally different from traditional security scanning, where coverage is determined by the rule set and remains constant across runs.

In practice, this distinction has important implications. Prompt-sensitive scanners are well-suited for exploratory analysis and for identifying classes of issues that are difficult to encode as static rules, such as business-logic flaws, authorization errors, or violations of implicit application invariants. In these cases, an LLM's ability to reason about intent, control flow, and higher-level semantics can surface issues that traditional static analysis may miss.

However, this same reliance on interpretation makes AI-only scanners less suited for automated or baseline security guarantees, where consistent and complete coverage is required regardless of user input, for well-defined vulnerability classes, such as secret exposure, unsafe rendering sinks, privacy leaks, and dependency-level risks. Coverage should not depend on how a prompt is phrased or which threats the user remembers to ask about. Rule-based scanners excel in this domain precisely because they decouple detection from intent, applying the same checks uniformly and deterministically across all code.

5. Design Implications

The results of this study suggest that no single technique is sufficient for securing AI-generated applications. Instead, effective security scanning requires a deliberate separation of responsibilities between deterministic analysis and probabilistic reasoning.

5.1 Baseline Security Should Be Deterministic

For certain classes of vulnerabilities, such as secret exposure, unsafe rendering primitives, privacy leaks, and dependency-level CVEs, the security scanning must provide baseline guarantees. These issues are well-understood, structurally identifiable, and occur frequently in practice. Detection should therefore be:



Deterministic



Reproducible



Independent of user intent or prompt phrasing

💡 5.2 LLMs Are Best Used for Higher-Level Reasoning

At the same time, our findings also highlight an area where LLMs provide clear value: business logic and intent-level vulnerabilities. These include issues such as:

Incorrect authorization logic

Violations of implicit invariants (e.g., "only admins should approve refunds")

Confusing or contradictory state transitions

Security assumptions embedded in application semantics rather than syntax

Such flaws are notoriously difficult to express as static rules, as they depend heavily on domain context and developer intent. In these cases, LLMs' ability to reason across multiple files, infer intent, and simulate user behavior can surface issues that traditional static analysis cannot. However, this strength depends on human guidance.

📦 5.3 Toward Hybrid Security Architectures for AI-Generated Code

Taken together, these implications point toward a hybrid architecture for securing AI-generated applications:

1

Deterministic scanners (SAST, SCA, privacy analysis) establish a consistent security baseline.

- 2 **LLMs augment** this baseline by reasoning about higher-level logic and proposing fixes.
- 3 **Feedback loops** ensure that AI-generated changes are validated by deterministic tools.

6. Threats to Validity and Scope

This study is an informal case study rather than a statistically exhaustive benchmark. Results reflect a specific class of applications and platform behavior at the time of evaluation.

While we applied controlled code variants to evaluate scanner robustness, these variants were intentionally simple and localized. More complex systems may introduce additional behaviors not covered here.

Finally, this work focuses exclusively on static and pre-deployment analysis. Runtime protections, dynamic analysis, and production monitoring are out of scope.

7. Conclusion

AI-generated code is rapidly becoming a first-class input to modern software systems. As this shift accelerates, the question is no longer whether AI can write code, but whether we can reliably and at scale reason about the security of that code.

This case study demonstrates that AI-only security scanning is fundamentally limited by nondeterminism, prompt sensitivity, and limited ecosystem awareness. While LLMs can be effective at identifying higher-level logic flaws and reasoning about intent, they cannot provide consistent baseline security guarantees on their own. Equivalent vulnerabilities expressed through minor syntactic variations were frequently missed,

and entire classes of risk—most notably dependency-level vulnerabilities—remained invisible.

In contrast, deterministic static analysis and dependency scanning consistently identified well-understood vulnerability patterns across all tested variants. These tools provide the reproducibility, coverage, and explainability required for automated security baselines and CI integration.

At the same time, this work also highlights where LLMs genuinely add value: in analyzing business logic, intent, and application-specific invariants that are difficult to encode as static rules. The most effective approach, therefore, is not to replace traditional security tooling with AI, but to compose them.

Taken together, our findings suggest a clear architectural principle for securing AI-generated applications:

- **Deterministic tools** should establish the security baseline
- **LLMs** should augment that baseline through reasoning and remediation

As AI systems increasingly participate in software creation, security systems must be designed not to mirror human intuition alone, but to combine probabilistic reasoning with infrastructure and tooling that provide consistency, isolation, and trust.

References

- [1] CVE-2024-24558 - Cross-site Scripting vulnerability in @tanstack/react-query-next-experimental
- [2] React Security Best Practices - dangerouslySetInnerHTML considerations
- [3] OWASP Top 10 Web Application Security Risks

Citation

APA

BibTeX

 Copy to Clipboard

Feng, D. (2026). Securing AI-Generated Code: Why Deterministic Tools Must Establish the Baseline. Replit Agent Infrastructure.

Replit Agent Infrastructure

© 2026 Replit. All rights reserved.